

CSCE 20104 – Programming Project 5
Due Date – April 10th, 2026 at 11:59 pm

1. Problem Statement:

The management team of *X² Robotics* has been receiving a significantly large number of support requests from customers. These requests, received daily via email, often vary in urgency, complexity, and safety concerns. For example, some tickets may report minor interface issues, while others may involve critical system failures affecting multiple users. As the number of tickets grows, it becomes more difficult for the company's support team to manually organize and prioritize them efficiently.

To try to solve this problem, the CEO of *X² Robotics* has hired you to develop a Ticket Management System. The purpose of this system is to store, sort and display support tickets based on different criteria such as customer name, priority level, date submitted. The system should assist support staff in identifying the tickets that require immediate attention and improve the overall efficiency of ticket handling.

In addition to managing ticket records, the application should allow users to apply different sorting methods and compare their performance. This will help determine which sorting approach is most suitable when handling small and large volumes of ticket data.

2. Design:

Task 1 - Support Ticket Data Structure

In order to facilitate the integration of the *X² Robotics* Ticket Management System with the company's current software, the CEO has asked you to store ticket information in a "Ticket" class with the following fields:

- Ticket ID – integer
- Customer Name – string
- Priority Level – string (low, medium, high, urgent)
- Date Submitted – three integers (month/day/year)
- Estimated Resolution Time – integer (days)

Your "Ticket" class should implement all of the methods we would expect to see in a standard C++ class (constructors, destructor, get/set methods, and print).

Task 2 - Support Ticket Operations

To simplify the management of support tickets, you have been asked to store all tickets in an array of Ticket objects. The operations you have been asked to implement are:

- Load Ticket Data: The system should allow users to read and store ticket records from an input file.
- Print Ticket Data: The system should be able to display all stored support records in tabular format.
- Add New Tickets: The system should allow users to add new support tickets through a menu-based input.
- Edit Existing Tickets: The system should allow the user to update details of an existing ticket by providing its Ticket ID.
- Delete Tickets: The system should allow the user to remove a ticket from the system using its Ticket ID.

To demonstrate the functionality of your code, ask your favorite generative AI to create 20 synthetic ticket records containing the data fields above. Copy/paste this information into a data file. Be sure to select a file format that you feel will be easy to read. For example, data fields on separate lines, or data fields separated by commas (CSV format).

Reminder: You will need to be very specific about what synthetic data fields you would like the AI to generate for you. Keep track of your generative AI prompts for your project report.

Task 3 – Adding Sorting Functionalities

The CEO was thrilled with your initial system, so they asked you to extend the software to support several sorting methods to make it easier to identify “problem” tickets. In particular, the CEO would like you to add operations so users can sort and display the tickets in the Ticket array according to one of the following ticket attributes:

- Customer Name (ascending alphabetical order)
- Priority Level (highest priority ticket first)
- Date Submitted (oldest ticket first)
- Estimated Resolution Time (longest time first)

The CEO is curious to see how different sorting methods work on Ticket data (and they want to test your programming ability) so they would like you to implement two “classic” sorting algorithms (e.g., bubble sort, insertion sort, merge sort, quick sort) and use one sorting method for the first two sorts above, and the other sorting method for the last two sorts.

Reminder: You are allowed to copy and modify any of the sorting code posted in the lecture notes or on the class website. Describe what code you used in your project report.

Task 4 – Performance Analysis

In order to demonstrate the excellence of your sorting implementation to the CEO, you need to add execution timers and data comparison counters to your sort functions. These comparison counters should only be incremented when you compare ticket attributes. You

can ignore loop counter comparisons. Your sorting operations should print elapsed time and the number of data comparisons after each request to sort the ticket data.

You need more than 20 ticket records to do any meaningful performance analysis. Ask your favorite generative AI to create 1000 synthetic ticket records containing the data fields above. Some generative AI systems will limit the amount of output they will produce per query so you may need to ask for 100-200 records multiple times to get the data you need.

Task 5 – User Interaction

Your final task is to design and implement a simple menu-based interface to your program that allows users to perform all of the ticket operations listed above. You are welcome to invent any command prompts you feel are easy to understand and read user inputs any way you wish. Describe your design decisions in your project report.

3. Implementation:

For this project you will be creating a Ticket class and a main program. If you use OnlineGDB or VS Code, you can create these programs using the methods described in the lab. To compile your programs on turing use the command:

```
g++ -Wall ticket.cpp project4.cpp -o project4.exe.
```

Once you have chosen the methods for each class it is a good idea to start with "skeleton methods" to get something to compile, and then add the desired code to each method incrementally writing comments, adding code, compiling, debugging, a little bit at a time. Once you have the methods implemented, it should be fairly simple to create a main program that calls these methods to complete your project.

Remember to use good programming style when creating your program; clear names for variables and constants, proper indenting for loops and conditionals, clear comments, etc. Also, be sure to save backup copies of your program somewhere safe. Otherwise, you may end up retyping your whole program if something goes wrong.

4. Testing:

Test your program to check that it operates correctly for all of the requirements listed above. See what happens if the user does not follow your instructions, or if they type invalid data. Try your program with several input values, and save your testing output in text files or as screenshots for inclusion in your project report.

Once your program is working properly, you need to demonstrate correctness on the department Linux server turing.csce.uark.edu. To do this, you need to copy your source code and data files to turing and compile and execute your program while running "script" as shown below.

```
script testing.txt
g++ -Wall ticket.cpp project4.cpp -o project4.exe.
./project4.exe
(type your inputs)
exit
```

If you are unable to login to turing to create the testing.txt file, you can use your personal computer instead. If you have a Mac, you can open a terminal window to execute the commands above. If you have a PC, you can install the Windows Subsystem for Linux (WSL) and run these commands in a terminal window.

5. Documentation:

When you have completed your program, write a short report using the “Programming Project Report Template” describing what the project objectives were, what you did, and the status of the program. The report template has a number of point form questions in each section. You should remove these questions from the report and replace them with your answers. Save this project report in a separate document to be submitted electronically.

Reminder: Include all of your generative AI prompts in your project report or a separate file that is clearly identified as your generative AI prompts.

6. Project Submission:

In this class, we will be using electronic project submission to make sure that all students hand their programming projects and labs on time, and to perform automatic plagiarism analysis of all programs that are submitted.

When you have completed the tasks above, create a folder on your computer called “your_last_name” and copy all of your C++ source code, input/output files, your testing.txt file, prompts.docx file, and project report into this folder. Then compress this file folder to create “your_last_name.zip”. Upload this zip file into Blackboard using the project upload link. We will download and decompress your zip file, and compile and run your code to verify correctness.

The dates on your electronic submission will be used to verify that you met the due date above. All late projects will receive reduced credit:

- 25% off if less than 1 day late,
- 50% off if less than 2 days late,
- 75% off if less than 3 days late,
- no credit if more than 3 days late.

You will receive partial credit for all programs that compile, even if they do not meet all program requirements, so handing projects in on time is highly recommended.

7. Academic Honesty Statement:

Students are expected to submit their own work on all programming projects, unless group projects have been explicitly assigned.

- Students are NOT allowed to distribute code to each other or copy code from another individual or website.
- Students are NOT allowed to use generative AI to create code UNLESS specific instructions on how to use generative AI are included in the project description.
- If generative AI is explicitly allowed for this project, all AI prompts should be saved in “prompts.docx” to be submitted with the rest of the project.
- Students ARE allowed to use any of the source code examples on the class website, or in the textbook in this project.
- Students ARE allowed to use online C++ documentation websites like <https://cplusplus.com/> for reference materials and C++ syntax examples.
- Students ARE allowed to ask the instructor and/or GTAs for assistance during lecture or office hours.

This course will be using highly effective program comparison software to detect plagiarism and unauthorized use of generative AI. Please do not be tempted to copy code from another student or from unauthorized sources. Violations of the policies above will be reported to the Academic Initiatives and Integrity office <https://honesty.uark.edu/> and may result in a ZERO on the programming project, an F in the class, or suspension from the university, depending on the severity of the violation and any history of prior violations.